RESEARCH DEVELOPMENT PACK

AI-Powered Precision Irrigation for Sustainable Vegetable Farming

From Research Proposal v2 to Executable Study: Roadmap, Literature Matrix, Methodology, Model Development Plan & OpenCode Build Prompts

---

Based on: "Research Proposal v2 — AI-Powered Precision Irrigation for Sustainable Vegetable Farming: Improving Water-Use Efficiency Through Low-Cost Edge Machine Learning"

Purpose: Bridge document between the existing proposal deck and a build-ready, defensible research study — no model built yet, no literature sourced yet.

Prepared: August 2026

Contents

> **Note on originality:** The literature entries in Section 3 are drawn from real, currently indexed studies (2021–2026) located via web search, summarized in original wording. Verify each citation's full bibliographic details (exact author list, volume/issue) directly from the linked source before submitting to your supervisor — indexed snippets do not always expose the full author string.

1. Purpose of This Document & How to Use It

Your current proposal deck ("Research Proposal v2") establishes the problem, aim, research questions, objectives, and a first-pass system architecture for a low-cost, edge-based, machine-learning-integrated precision irrigation system for vegetable farming. It is strong on framing but — by your own note — has **no literature sourced yet** and **no model built yet**. This document is the bridge between that proposal and an executable, defensible study.

Read it in this order:

1. **Section 2** tells you the sequence of work, phase by phase, with a suggested timeline — this is your "what do I do Monday morning" answer.
2. **Section 3** gives you a literature matrix to seed your related-work chapter, organized by the four themes your own slides already named (ML irrigation prediction, edge/IoT hardware, evapotranspiration/soil-moisture modeling, farmer adoption).
3. **Section 4** is the full methodology write-up — research design, data collection protocol, experimental design, and the qualitative adoption study — ready to adapt into your thesis/proposal document.
4. **Section 5** is the technical model-development plan: data schema, feature engineering, candidate models, evaluation strategy, and repo structure.
5. **Section 6** gives you ready-to-paste prompts for **OpenCode** (or any agentic coding CLI) to scaffold the actual codebase — data pipeline, training pipeline, and edge inference harness — so you go from "no model" to a working, evaluated baseline quickly.

2. Research Roadmap — What To Do Next

Your proposal is currently at the **conceptual/design stage**. The path from here to a defensible study runs through eight phases. Phases 1–3 can run partly in parallel; phase 4 (model development) does **not** need to wait for the full field deployment — you should build and validate the ML pipeline on a synthetic or public dataset first (Section 5 explains why and how), then swap in your real sensor data once collection begins.

Phase	Focus	Key activities	Suggested duration
Phase 0	Literature consolidation	Expand Section 3's matrix to 25–35 sources; identify your 3–5 closest "sibling" studies (this pack flags the closest ones already); write the related-work narrative and firm up the stated research gap.	2 weeks
Phase 1	System & sensor design finalization	Lock the sensor list (soil moisture, temperature, humidity, water flow — per your slide 8/12) and the edge board (ESP32 / Raspberry Pi class device); produce a bill of materials and a wiring/architecture diagram; decide vegetable crop(s) and plot layout.	2 weeks
Phase 2	Instrument & protocol design	Finalize the data-collection protocol (Section 4.3), the farmer interview guide and usability survey (Section 4.7), and get ethics/IRB or institutional sign-off if required.	2 weeks
Phase 3	Pilot data collection	Deploy sensors on a small pilot plot (or one bed) for 2–3 weeks purely to catch hardware/logging bugs before the full crop cycle starts.	2–3 weeks
Phase 4	ML pipeline build (parallel-startable)	Build the full training/evaluation pipeline now on synthetic + any public soil-moisture/irrigation datasets you can access (Section 5); this de-risks the modeling work so it isn't blocked on field data. Use the OpenCode prompts in Section 6 to scaffold this immediately.	3–4 weeks, overlapping Phases 1–3
Phase 5	Full-season field deployment	Run the sensor network + automated irrigation across a full vegetable crop cycle, in parallel with a conventional-irrigation control plot; log continuously.	1 full crop cycle (typically 8–14 weeks for most vegetables)
Phase 6	Model retraining on real data + edge deployment	Retrain/fine-tune RF/XGBoost on the real sensor dataset collected in Phase 5; export and benchmark the model on the edge device (latency, memory, power draw).	2–3 weeks
Phase 7	Farmer adoption study	Run semi-structured interviews and the TAM-based usability survey with the farmers who hosted or observed the trial (Section 4.7); this can start as soon as farmers have seen the system operate, even before full harvest data is in.	2–3 weeks
Phase 8	Analysis, triangulation & write-up	Combine quantitative results (WUE, yield, model accuracy) with qualitative findings (adoption barriers) using the joint-display technique described in Section 4.1; write the discussion, limitations, and recommendations.	3–4 weeks

Immediate next 3 actions (this week):
1. Pick the specific vegetable crop(s) and confirm plot access — this determines your growth-cycle length and therefore your whole timeline.
2. Order/confirm sensor + edge hardware (see the bill-of-materials basis in Section 3, entry on the low-cost modular system) so Phase 1–3 aren't blocked on procurement.
3. Run the Section 6 "Prompt 1" through OpenCode to scaffold the ML repo and start Phase 4 in parallel — you do not need real data to start this.

3. Literature Matrix

16 studies (2021–2026), organized by the four themes already named in your proposal deck's "Literature Review" slide. Findings are paraphrased from indexed abstracts/full texts; use the link to pull the exact citation format your institution requires. "Relevance / gap to this study" is the column to lift directly into your related-work chapter.

Theme A — Machine Learning for Irrigation Prediction & Scheduling

Source	Focus	Key finding	Relevance / gap to this study
"Intelligent Irrigation Scheduling of Selected Agricultural Crops: A Machine Learning Approach" — Academia.edu, 2025	DT, RF, Naive Bayes	Trained on a large synthetic/secondary dataset (~150,000 records, 197 crop types, 10 soil classes) to select among 10 irrigation methods; decision tree reached near-perfect accuracy on held-out data.	Shows large-scale classification is feasible, but on secondary/synthetic data rather than field sensors. Gap this study fills: real sensor-collected data from an actual vegetable plot, not a pre-built dataset.
"Smart IoT-driven precision agriculture: land mapping, crop prediction, and irrigation system" — PLOS ONE / PMC, 2025	RF, XGBoost, LightGBM, CatBoost + fuzzy logic	Benchmarked eight ML models for crop suitability and yield prediction (RF ~97% for crop classification, ~96% for yield), paired with a solar-powered fuzzy-logic irrigation controller.	Provides a template for multi-model benchmarking (this study's own RF-vs-XGBoost comparison mirrors this design). Gap: no reported water-use-efficiency figures or farmer-facing evaluation.
"Sustainable Irrigation System for Farming Supported by Machine Learning and Real-Time Sensor Data" — PMC, ~2021	DT, RF, Neural Network, SVM	Compared four algorithms for predicting the best time of day to irrigate from real-time sensor data; Random Forest was the top performer at ~84.6% accuracy.	Directly comparable RF baseline for irrigation-timing prediction — a realistic accuracy benchmark to cite when reporting this study's own RF results.
"On-device AI for climate-resilient farming with intelligent crop yield prediction using lightweight models on smart agricultural devices" — Scientific Reports, 2025	Edge-deployed Random Forest	Random Forest classifier embedded directly in a consumer smart-display device for water-use optimization, no cloud dependency, ~90% accuracy.	Closest technical precedent for this study's "edge device runs the model" design. Gap: does not jointly evaluate water-use efficiency *and* farmer adoption, which this study proposes to do.
"Scalable machine learning framework for adaptive irrigation management of maize and soybean in the U.S. Midwest" — ScienceDirect, 2025	DT, RF, Gradient Boosting, XGBoost	Predicted soil water depletion and validated ML-driven irrigation recommendations directly against the FAO-56 spreadsheet method in field trials; XGBoost achieved $R^2 \geq 0.72$ and matched agronomic-standard recommendations within ±4mm.	Gold-standard validation approach — comparing the ML model's output against a recognized agronomic baseline (FAO-56 / Penman-Monteith) rather than only against ground truth. This study should replicate that validation logic at smallholder/vegetable scale.
"Machine Learning-Based Basil Yield Prediction in IoT-Enabled Indoor Vertical Hydroponic Farms" — arXiv, 2025	RF, XGBoost, hybrid RF-XGBoost	Reviews evidence that RF is robust on noisy IoT/satellite data but has interpretability and computational-efficiency limits on resource-constrained edge hardware.	Directly informs this study's model-selection rationale (Section 5.3) and justifies benchmarking a lighter XGBoost variant alongside RF for edge deployment.

Theme B — Low-Cost Edge Computing & IoT Hardware for Irrigation

Source	Focus	Key finding	Relevance / gap to this study
"An IoT Low-Cost Smart Farming for Enhancing Irrigation Efficiency of Smallholder Farmers" — ResearchGate, 2022	Cloud-based IoT sensing	Combines soil moisture, temperature, humidity, water-level and water-flow sensors with the Rawls & Turq formulas for water-quantity computation, plus cloud-hosted ML.	The sensor suite matches this study's proposed architecture almost exactly. Gap: the system is cloud-dependent, which this study avoids by pushing inference to the edge for rural-connectivity resilience.
"TinyML-Enabled IoT for Sustainable Precision Irrigation" — ResearchGate, 2026	On-device AI / TinyML	Proposes a practical blueprint for on-device inference specifically to bridge the technology divide where cloud/internet dependency is unreliable in rural areas.	Directly justifies the "Edge Device — Data Collection & Local Processing" tier in this study's proposed architecture (proposal slide 8); a strong citation for the "why edge, not cloud" argument.
"Deploying Low-Cost and Full Edge-IoT/AI System for Optimizing Irrigation in Smallholder Farmer	Fog-IoT/AI "irrigation in a box"	Describes a plug-and-sense, low-cost, open-source irrigation control system aimed explicitly at smallholder communities, using soil sensors to reduce water use.	The closest existing prototype in spirit to this study's proposed system — useful as a direct comparator when costing your own bill of materials.

Communities" — IOS Press, 2022			
"A low-cost modular precision irrigation system for resource-constrained environments" — ScienceDirect, 2026	ESP32 master-slave + custom valve	ESP32-based sensor network with a custom-built low-cost electric valve (cutting hardware cost roughly in half, ~$50 to ~$25), tested across five farms in Northern Uganda over 100 weeks.	The strongest available cost/hardware benchmark and multi-farm field-deployment precedent. Use this as the baseline for your own bill-of-materials and for framing "low-cost" claims with a concrete dollar figure.
"Smart drip irrigation systems using IoT: a review of architectures, machine learning models, and emerging trends" — Discover Agriculture (Springer), 2025	Review	Identifies rural connectivity, high upfront hardware cost, and energy constraints as the three recurring barriers to smallholder IoT-irrigation adoption, and calls for local edge processing as a mitigation.	Supports the research-gap statement almost word-for-word with your proposal's own framing — strong citation for your introduction/background section.

Theme C — Evapotranspiration & Soil-Moisture Modeling

Source	Focus	Key finding	Relevance / gap to this study
"A deep learning based framework for enhanced reference evapotranspiration estimation" — Scientific Reports, 2025	Deep learning vs. Penman-Monteith	Shows deep-learning models can approximate FAO-56 Penman-Monteith reference evapotranspiration using fewer meteorological input variables than the full physical formula requires.	Useful for justifying feature-engineering choices if some meteorological sensors are unavailable/unaffordable — you don't necessarily need the full Penman-Monteith input set.
"Soil moisture forecast for smart irrigation: the primetime for machine learning" — ScienceDirect, 2022	Data-driven soil-moisture forecasting	Found that past soil-moisture history plus a precipitation forecast outperformed models that included full evapotranspiration and soil-hydraulic domain features.	Directly informs feature-selection in Section 5.2 — a data-driven, historical-soil-moisture-led feature set may be simpler and just as effective as computing full evapotranspiration on a low-cost edge device.
"Evaluating the accuracy of machine learning models in predicting soil moisture, actual evapotranspiration, and biomass... for watermelon" — ScienceDirect, 2025	RF vs. ANN, real vegetable/melon crop	Compared RF and ANN across four irrigation treatments (100%, 100%+mulch, 50%, 50%+mulch) on a real vegetable-family crop; ANN outperformed RF (R^2 up to 0.92 for biomass).	The closest direct crop-analogy comparator available — a vegetable-family crop, real field treatments, and reported R^2 ranges you can use as realistic performance targets. Also models a usable treatment-arm design (full vs. reduced irrigation) for your own experiment.

Theme D — Farmer Adoption of Precision/Smart Agriculture

Source	Focus	Key finding	Relevance / gap to this study
"The Farm-Level Economic and Environmental Benefits of Precision Agriculture Technology Adoption: A Meta-Analysis of Global Evidence" — MDPI Sustainability, 2025	Meta-analysis, 85 studies / 1,472 farms	Benefits of precision-agriculture technology scale with farm size; smallholders face steeper adoption barriers, risking a "digital divide" within agriculture.	Quantitative backdrop for your adoption sub-question — supports the argument that low-cost design (not just accuracy) is a legitimate, evidence-based research contribution.
"Precision agriculture technology adoption: a qualitative study of small-scale commercial 'family farms' in the North China Plain" — Precision Agriculture (Springer), 2021	Qualitative interviews	Cost, suitability for small-scale operation, and trust were the dominant barriers; recommends low-cost precision-agriculture tools purpose-built for smallholders.	A direct methodological template for this study's own farmer interview design and thematic coding approach (Section 4.7).
"Factors Influencing Precision Agriculture Technology Adoption Among Small-Scale Farmers in Kentucky and Their Implications for Policy and Practice" — MDPI	Survey, TAM-based	Cost (~20% of respondents), complexity (~15%), and uncertain profitability (~12%) were the top-ranked adoption barriers in a Technology Acceptance Model (TAM)-based survey.	Gives this study's usability survey a validated TAM-based structure, and a published barrier-ranking benchmark to compare your own farmer-survey results against.

How to grow this matrix: for your formal literature review, aim for 25–35 sources. Use each theme above as a search seed (e.g., "XGBoost soil moisture vegetable," "TinyML irrigation Raspberry Pi," "technology acceptance model smart irrigation smallholder") and prioritize studies published in the last 3 years plus 2–3 foundational/highly cited older studies per theme.

4. Research Methodology

4.1 Research Design

This study adopts an **explanatory sequential mixed-methods design**: a quantitative phase (system deployment, ML model development, and water-use/yield measurement) runs first and largely in parallel with an embedded qualitative phase (farmer interviews and usability surveys) that begins once farmers have observed the system operating. The two strands are combined at the end using a **joint-display technique** — a results table that places the quantitative finding (e.g., "AI-scheduled plot used 28% less water") directly beside the qualitative finding that explains or qualifies it (e.g., "farmers distrusted the automated valve at first and manually overrode it for the first two weeks"). This directly answers your proposal's own framing: moving beyond "how accurate is the AI?" to "does it actually save water, support production, and get adopted?"

4.2 System Architecture

The architecture follows the pipeline already sketched in your proposal (slides 8 and 12), formalized here as five layers:

1. **Sensing layer** — soil moisture, temperature, humidity, and water-flow sensors installed in and around the vegetable plot, sampling at a fixed interval (e.g., every 15–30 minutes).
2. **Edge layer** — a low-cost microcontroller/single-board computer (ESP32-class or Raspberry-Pi-class) that collects sensor readings, runs the trained ML model locally, and logs data — no dependency on continuous internet connectivity.
3. **Decision layer** — the trained model (Random Forest or XGBoost, see Section 5) converts current + recent sensor readings into an irrigation decision: whether to irrigate, and for how long.
4. **Actuation layer** — an automated pump/solenoid or low-cost electric valve executes the irrigation decision (drip or sprinkler, matching your crop and plot).
5. **Feedback layer** — post-irrigation sensor readings feed back into the system, closing the loop and providing the time-series data used to retrain/validate the model.

> **Design decision to make in Phase 1:** ESP32 (cheaper, lower power, harder to run a full RF model on directly — often used with a lightweight/quantized model or as a sensor-and-relay node reporting to a Raspberry Pi) vs. Raspberry Pi (more compute headroom, higher cost/power draw, easier full RF/XGBoost inference). The low-cost modular system in the literature matrix (Theme B) used an ESP32 master-slave design with a companion mobile app — a reasonable template if budget is the binding constraint.

4.3 Data Collection Protocol

Variable	Instrument	Frequency	Notes
Soil moisture (%)	Capacitive soil moisture sensor, at 2 depths if feasible	Every 15–30 min	Primary predictor per Theme C literature — most predictive single feature.
Air temperature (°C)	DHT22 / SHT-class sensor	Every 15–30 min	Co-located with humidity sensor.
Relative humidity (%)	DHT22 / SHT-class sensor	Every 15–30 min	—
Water flow / volume applied	Flow meter on irrigation line	Per irrigation event	Needed to compute water-use efficiency (WUE).
Rainfall	Tipping-bucket sensor or nearest weather API	Continuous / daily	Use API as fallback if a physical rain gauge isn't feasible.
Crop growth stage	Manual observation log (photo + stage label)	Weekly	Needed to compute crop coefficient (Kc) if you benchmark against Penman-Monteith.
Yield at harvest	Manual weighing per plot	End of cycle	Primary agronomic outcome measure.

Run the full protocol on both the **AI-irrigated treatment plot** and a **conventionally irrigated control plot** of the same crop, planted at the same time, under comparable soil/light conditions — this pairing is what lets you attribute differences in WUE and yield to the irrigation method rather than to seasonal variation.

4.4 Quantitative Analysis Plan

- **Water-use efficiency (WUE):** crop yield (kg) ÷ total water applied (m^3), compared between treatment and control plots.
- **Model performance:** reported per Section 5.4 (RMSE/MAE/R^2 for regression framing of "how much water to apply"; accuracy/F1/precision/recall if framed as an irrigate/don't-irrigate classification).
- **Statistical comparison:** depending on replicate count, use a paired t-test or Mann-Whitney U test to compare WUE and yield between treatment and control; report effect size, not only p-value, given likely small plot-level sample sizes.

4.5 Sampling & Site Considerations

Because this is a single-site or few-site field study (not a large-N agronomic trial), be explicit in your write-up about the resulting limits on statistical generalizability — this is normal for a prototype/feasibility study and should be framed as such rather than defended as if it were a multi-farm randomized trial. If more than one host farm is available, treat each farm as a replicate block and note soil-type/microclimate differences between them.

4.6 Qualitative Study Design — Farmer Adoption

Two instruments, both grounded in the literature matrix (Theme D):

- **Semi-structured interviews** (10–15 minutes, 8–15 farmers) covering: prior irrigation practice, first impressions of the automated system, trust in the AI's decisions, perceived cost/benefit, and what would need to change for them to adopt it long-term. Use an inductive thematic-coding approach (code the transcripts, group into themes, don't force-fit to a predetermined framework).
- **Structured usability/acceptance survey**, built on the Technology Acceptance Model (TAM) — perceived usefulness, perceived ease of use, and behavioral intention to adopt — mirroring the Kentucky study in the literature matrix, so your barrier rankings (cost / complexity / trust / profitability) are directly comparable to a published benchmark.

4.7 Ethical Considerations

- Obtain informed consent from all interviewed/surveyed farmers; explain data use plainly, in their working language.
- If your institution requires it, secure ethics/IRB approval before any farmer-facing data collection begins — build this into Phase 2 of the roadmap, since approval timelines are often the true critical path.
- Anonymize farmer names and exact plot locations in any published output.
- Be transparent with host farmers that this is a prototype — do not let them rely on it as their sole irrigation method for a commercially critical crop without a manual override always available.

4.8 Validity, Reliability & Limitations

- **Internal validity:** strengthened by the paired treatment/control plot design; weakened by any uncontrolled differences between plots (soil heterogeneity, shading) — document these.
- **External validity:** limited by single-site/single-season scope; state this explicitly rather than over-claiming generalizability to other crops, climates, or farm scales.
- **Reliability:** sensor drift and calibration error should be checked periodically (e.g., manual soil-moisture spot checks against the sensor reading) and reported.
- **Common-method limitation:** the qualitative adoption findings come from farmers who watched a researcher-run pilot, not from farmers who purchased and self-installed the system — note this as a boundary condition on the adoption conclusions.

5. Machine Learning Model Development Plan

This is the concrete technical plan for turning "no model built yet" into a working, evaluated, edge-deployable model. The key move: **do not wait for field data to start building.** Build and validate the full pipeline now against a synthetic dataset generated from realistic ranges (Section 5.1 gives the schema), then swap in real sensor data once Phase 5 collection is underway. This is exactly what Prompt 1 in Section 6 sets up.

5.1 Data Schema

```
timestamp            | datetime  | ISO-8601, sampled every 15-30 min
plot_id              | string    | treatment | control
soil_moisture_pct    | float     | 0-100
soil_temp_c          | float     | typically 10-40
air_temp_c           | float
air_humidity_pct     | float     | 0-100
rainfall_mm_24h      | float     | rolling 24h rainfall
days_since_planting  | int
crop_growth_stage    | category  | e.g. establishment/vegetative/flowering/maturity
water_applied_l      | float     | logged per irrigation event (target/label source)
irrigated_next_flag  | bool      | derived label: was water applied in the next window?
```

Two label framings are worth building in parallel — pick one as primary, keep the other as a stretch goal:

- **Classification framing:** predict `irrigate / don't irrigate` for the next time window (mirrors the "84.6% accuracy" RF benchmark study in Theme A — a realistic target to beat or match).
- **Regression framing:** predict `liters of water to apply` directly (mirrors the soil-water-depletion regression study in Theme A — richer output, harder to get right with limited data).

5.2 Feature Engineering

- Lagged soil moisture (t-1, t-2, rolling 3-hour mean) — the Theme C soil-moisture-forecast study found past soil moisture more predictive than full evapotranspiration inputs, so prioritize this over trying to compute full Penman-Monteith on the edge device.
- Rolling rainfall sum (past 24h, past 72h).
- Time-of-day and day-of-crop-cycle (captures diurnal and growth-stage effects).
- Crop growth stage, one-hot or ordinal encoded.
- Optional/stretch: a simplified evapotranspiration proxy (temperature + humidity based, not full Penman-Monteith) if you want a physically-grounded feature without the full instrument set the FAO-56 method requires.

5.3 Candidate Models & Why

Model	Role	Rationale
Rule-based / threshold baseline	Sanity-check floor	"Irrigate if soil moisture < X%" — every ML model must beat this to justify its added complexity. Always report this number.
Random Forest	Primary candidate	Consistently the strongest classical model across the Theme A literature (84–97% accuracy range depending on task/dataset); robust to noisy IoT sensor data; interpretable via feature importance.
XGBoost	Primary candidate	Matches or beats RF on regression-style tasks (soil-water-depletion study, Theme A) and is generally lighter/faster at inference — relevant for edge deployment.
Linear/logistic regression	Secondary baseline	Cheap interpretability check; if it performs nearly as well as RF/XGBoost, that's a meaningful finding about how "AI" this problem actually needs to be — worth reporting honestly.
Small ANN (stretch goal)	Exploratory	The watermelon study (Theme C) found ANN beat RF on a real vegetable-family crop — worth trying if time allows, but not at the cost of the core RF/XGBoost comparison.

5.4 Evaluation Strategy

- **Split:** time-based split (not random shuffle) — train on the first ~70% of the crop cycle chronologically, validate on the next ~15%, test on the final ~15%. Random shuffling would leak future information into training and inflate accuracy.
- **Metrics — classification framing:** accuracy, precision, recall, F1 (report per-class, since "don't irrigate" will likely be the majority class and accuracy alone can mislead).

- **Metrics — regression framing:** RMSE, MAE, R^2 — directly comparable to the Theme A/C benchmarks ($R^2 \geq 0.72$ for the XGBoost soil-water-depletion study; R^2 0.75–0.92 range for the watermelon ANN/RF study).
- **Cross-validation:** use k-fold (k=5) time-series-aware cross-validation (e.g., `sklearn.model_selection.TimeSeriesSplit`) for hyperparameter tuning, not standard k-fold.
- **Explainability:** compute SHAP values or built-in feature importance for the winning model — you already flagged "Gini > 0.1" feature importance filtering on slide 12, keep that as your feature-selection threshold.
- **Agronomic benchmark:** where feasible, compare your model's irrigation recommendation against a simplified Penman-Monteith/FAO-56 calculation for the same period, following the Theme A "Scalable ML framework" study's approach of validating against an agronomic-standard method, not only against held-out ground truth.

5.5 Edge Deployment Strategy

- Export the trained scikit-learn/XGBoost model via `joblib`/ONNX; for microcontroller-class devices (ESP32) consider `micromlgen` or `m2cgen` to transpile a small tree ensemble into C code, or fall back to running the trained model on a companion Raspberry Pi that the ESP32 reports sensor data to.
- Benchmark on-device: inference latency, memory footprint, and power draw — report these, since "low-cost and edge-deployable" is a core claim of your proposal and needs its own evidence, not just model accuracy.
- Build a simple safety fallback: if the model is unavailable or sensor readings are out-of-range, fall back to the rule-based threshold baseline rather than failing irrigation entirely.

5.6 Suggested Repository Structure

```
precision-irrigation-ml/
├── data/
│   ├── raw/                # untouched sensor exports / synthetic generator output
│   ├── processed/          # cleaned, feature-engineered datasets
│   └── synthetic_generator.py
├── notebooks/
│   └── 01_eda.ipynb
├── src/
│   ├── data_pipeline.py    # ingest, clean, merge, feature engineer
│   ├── train.py            # train RF / XGBoost / baselines
│   ├── evaluate.py         # metrics, SHAP, agronomic-benchmark comparison
│   ├── tune.py              # TimeSeriesSplit + grid/optuna search
│   └── export_edge_model.py # export to ONNX / m2cgen / joblib
├── edge/
│   ├── firmware/            # ESP32 / microcontroller sensor+relay code
│   └── inference_service.py  # Raspberry-Pi-side inference + actuation logic
├── dashboard/
│   └── app.py               # simple monitoring dashboard (Streamlit/Flask)
├── tests/
├── requirements.txt
└── README.md
```

6. OpenCode Build Prompts

These are copy-paste-ready prompts for **OpenCode** (or any comparable agentic coding CLI). They follow the repo structure from Section 5.6 and can be run one after another. Run Prompt 1 first — it does not require your real field data and gets a working, evaluated baseline model in place immediately, unblocking Phase 4 of the roadmap while field deployment (Phases 1–3, 5) is still in progress.

Prompt 1 — Scaffold the ML pipeline & get a synthetic-data baseline running

PASTE INTO OPENCODE

```
You are building the machine-learning pipeline for a research project: "AI-Powered Precision
Irrigation for Sustainable Vegetable Farming — Improving Water-Use Efficiency Through Low-Cost
Edge Machine Learning." No real sensor data exists yet, so start with a realistic synthetic
dataset and build the full pipeline against it.

GOAL
Produce a working, evaluated baseline that predicts irrigation need from soil/weather sensor
readings, structured so real field data can be swapped in later with no code changes beyond
the data-loading step.

TECH STACK
Python 3.11, pandas, numpy, scikit-learn, xgboost, shap, matplotlib, pytest. Use a src/ layout.

TASKS
1. Create the repo structure:
   data/{raw,processed}, notebooks/, src/, edge/, dashboard/, tests/, requirements.txt, README.md

2. data/synthetic_generator.py
   Generate ~90 days of 30-minute-interval sensor data for TWO plots ("treatment", "control"):
   soil_moisture_pct, soil_temp_c, air_temp_c, air_humidity_pct, rainfall_mm_24h,
   days_since_planting, crop_growth_stage (establishment/vegetative/flowering/maturity based on
   days_since_planting), water_applied_l, irrigated_next_flag.
   Model soil moisture as decaying over time (faster decay at higher temp/lower humidity) and
   jumping up after irrigation or rainfall events. Add realistic sensor noise. Save to
   data/raw/synthetic_sensor_log.csv. Make the generator's random seed configurable for
   reproducibility.

3. src/data_pipeline.py
   Load raw data, clean it (handle missing/out-of-range sensor values), engineer features:
   lagged soil moisture (t-1, t-2, rolling 3h mean), rolling rainfall sums (24h, 72h),
   time-of-day, crop growth stage encoding. Output data/processed/features.csv.
   Write this so a future real-data CSV with the same column names drops in with zero code
   changes.

4. src/train.py
   Implement a chronological (NOT random) train/val/test split. Train four models for the
   classification framing (predict irrigated_next_flag): a threshold-rule baseline
   (soil_moisture_pct < X), logistic regression, Random Forest, and XGBoost. Use
   sklearn.model_selection.TimeSeriesSplit for cross-validation during hyperparameter tuning
   (put this in src/tune.py, keep it simple — RandomizedSearchCV is fine). Save trained models
   to models/ via joblib.

5. src/evaluate.py
   Report accuracy, precision, recall, F1 (per class) for every model on the held-out
   chronological test set, in a single comparison table printed to console and saved as
   results/model_comparison.csv. Compute SHAP values or built-in feature_importances_ for the
   best-performing model and save a bar chart to results/feature_importance.png.

6. src/export_edge_model.py
   Export the winning model via joblib, and also attempt an ONNX export (skl2onnx). Print the
   resulting file size in KB — this matters for edge deployment feasibility.

7. tests/
   Add at least 3 pytest tests: one for the synthetic generator's output schema, one for the
   feature pipeline (no NaNs/infinities after processing), one that the chronological split
   never leaks future rows into the training set.

8. README.md
   Document how to regenerate data, run training, and interpret the comparison table. Include a
   clearly marked section: "Swapping in real field data" — explain exactly what CSV columns and
   format a future real dataset must match to work with this pipeline unchanged.

CONSTRAINTS
- No internet-dependent calls in the pipeline itself (must run fully offline once dependencies
  are installed).
- Keep every script runnable standalone via `python -m src.
```